

# RBE4701 – Bomberman Agent Report

Brian Grande  
Robotics Engineering  
Worcester Polytechnic Institute  
Worcester, MA  
bmgrande@wpi.edu

Kylie Herbstzuber  
Robotics Engineering  
Worcester Polytechnic Institute  
Worcester, MA  
kmherbstzuber@wpi.edu

Dhruv Madan  
Robotics Engineering  
Worcester Polytechnic Institute  
Worcester, MA  
dmadan@wpi.edu

**Abstract**—This report presents the design and evaluation of our AI agent for the Bomberman environment. We describe the structure of our approach, break down interesting components such as path planning and hazard avoidance, and report experimental survival rates across scenarios.

**Index Terms**—Artificial Intelligence, State Machines, Reinforcement Learning, Bomberman, Robotics

## I. INTRODUCTION

Provide context for the project: what the Bomberman environment is, the rules of the game, and why the assignment matters for learning AI decision-making.

## II. STRUCTURE OF THE APPROACH (STATE MACHINE)

Describe the overall design of your agent. What states does it use (e.g., Explore, Evade, Exit)? How do transitions occur between states? Optionally, include a diagram showing the state machine.

### A. Variant 1:

The structure of variant one is based on a state machine with two states. The agent has the state options of search / explore or exit; these states are determined by the agent's position in the map. If the agent is not at the exit location, it will continue to explore. If it is at the exit location, it will exit the map.

### B. Variant 2:

### C. Variant 3:

### D. Variant 4:

### E. Variant 5:

## III. INTERESTING COMPONENTS OF THE APPROACH

Discuss the most important parts of your solution.

### A. Variant 1

1) *Path Planning*: variant 1 uses a simple A\* path planner to determine the shortest route to the exit from the character start location.

2) *Hazard Avoidance*: Since Variant 1 uses a simple A\* path planner while also not having to deal with monsters, Hazard avoidance is built into the map itself as walls are looked at as non-accessible locations and are not considered as valid paths. This inherently becomes the hazard avoidance.

3) *Decision Making*:

4) *Other Features*:

### B. Variant 2

1) *Path Planning*: In Variant 2, the agent uses simulated Annealing rather than a fixed search like A\*. The idea is to treat finding a good path to exit as an optimization problem. The agent starts with an initial candidate path (for example, a greedy path in this case) and then repeatedly makes small random modifications to it. If the new path is better (shorter, safer, or closer to the exit), it is then accepted. If it is much worse, it may still be accepted with a probability that decreased as the "temperature" lowers over time. This mechanism allows the agent to escape local minima and gradually converge towards a near-optimal route to the exit.

2) *Hazard Avoidance*: Hazards, such as monsters are encoded into the cost function used by simulated annealing. Paths that pass too close to an active bomb or projected monster position are penalized with higher costs. During optimization, the annealing process naturally pushes the agent towards candidate paths that are minimized to expose the agent to hazards. Due to the algorithm accepting occasional worse moves early on, it can still explore alternate safe corridors even if they initially look promising.

3) *Decision Making*: The annealing schedule controls the balance between exploration and exploitation. At high temperature, the agent explores many different random path variations, which helps it find escape routes even in crowded or dangerous maps. As the temperature cools, the agent becomes more conservative and locks into the best low-risk path found so far. This trade-off lets the agent both escape quickly and maintain survival, since unsafe but short paths are eventually rejected in favor of safer alternatives.

4) *Other Features*: Several enhancements were added to a "standard simulated annealing" to make it effective in the Bomberman domain"

- **Initial solution bias**: The starting path is seeded with a starting heuristic in this case a greedy move towards the exit. Instead of being truly random, which speeds up convergence.
- **Cooling schedule**: The temperature decreases gradually each turn, balancing early exploration with late stability.
- **Cost shaping**: The cost function includes both path length and hazard penalties, ensuring that survival is prioritized over raw speed.

Overall, Variant 2 demonstrates how a stochastic optimization method like simulated annealing can yield us a robust escape strategy in the Bomberman game.

### C. Variant 3

The final agent for variant 3 utilizes the same strategy as variant five due to variant 5's superior decision-making ability. Below is an explanation of how variant 3's original approach was done. This agent did not end up working effectively due to a few issues. Through testing it was determined that it would succeed approximately 50% of the time which was not confident enough for the submission. One issue is that there is no intermediate between search and evade which can lead the agent into dead ends in small spaces.

1) *Path Planning*: In Variant 4, the agent uses A\* search as its primary method of path planning. It computes paths from the agent's position to the goal (exit). Movement is allowed in 8 directions (including diagonals). The A\* heuristic is a modified Manhattan distance that also includes a penalty for proximity to monsters. This means the agent does not simply minimize distance to the goal, but it also minimizes the distance while also preferring safer routes.

2) *Hazard Avoidance*: The agent builds a danger map to help with hazard avoidance. Cells with a monster, the monster's predicted straight-line trajectory, and adjacent junction cells are all marked as dangerous. All cells are then checked around the agent; if any are dangerous, the agent switches into evade state. In evade mode, the agent looks at the 8 neighboring cells and chooses the one with the lowest danger score. This logic allows the agent to proactively move away from monsters and avoid being caught in their line of travel.

3) *Decision Making*: The decision logic is a state machine with two states: - Search: If the local neighborhood is safe, the agent uses A\* to plan toward the exit. - Evade: If the danger map shows a nearby hazard, the agent prioritizes safety by moving to the lowest-risk neighboring cell.

4) *Other Features*: Predictive monster modeling: Monsters are not only marked at their current location, but their future path is traced until blocked, so the agent avoids not just where monsters are, but where they're heading.

### D. Variant 4

The final agent for variant 4 utilizes the same strategy as variant 5 due to variant 5's superior decision-making ability. Below is an explanation of how variant 4's original approach was done.

1) *Path Planning*: Before using Variant 5, Variant 4 used to use A\* as the method of path planning, using a calculated heuristic that was made up of the sum of

2) *Hazard Avoidance*: Explain how your agent detects bombs and monsters and avoids them.

3) *Decision Making*: Explain how your agent balances escaping quickly with survival.

4) *Other Features*: Describe any special techniques you added.

### E. Variant 5

1) *Path Planning*: Variant 5 treats our navigation as a Markov Decision Process (MDP), and computes a policy using value iteration. At each of our ticks, the agent first builds a backward flood map from the exit to every cell, proving a distance to exit heuristic. Each open grid cell is modeled as a state, and actions include moving to any of the 8 neighbors or staying put. The transition model is stochastic: the intended move succeeded with 80% probability, while the remaining 20% is distributed among other legal moves. The reward function combines a negative step cost, a strong positive reward for reaching the exit, and distance-based shaping to encourage progress.

2) *Hazard Avoidance*: Hazard handling in Variant 5 is two-tiered:

- **Immediate hazards**: A tile is treated as highly negative if it contains an active monster, or if it is adjacent to a monster or reachable by a monster in one step. These states receive a large penalty in the reward function, making them [hopefully] virtually inaccessible.
- **Risk map**: Beyond immediate threat, the agent constructs a probabilistic "danger field" by diffusing monster positions several steps into the future with exponential decay. This produces a smooth risk distribution, penalizing states more heavily when they are closer to a likely monster path.

Together, these mechanisms prevent the agent from walking into certain death and guide it away from likely hazards.

3) *Decision Making*: Decision making is framed as expected return on the maximization under the MDP. The value iteration policy already balances survival and progress by trading off exit proximity against hazard penalties. On top of this, a lightweight "safety net" heuristic evaluates the chosen next step: if it leads into unsafe state, the agent instead selects the locally safest option by scoring neighboring tiles on closeness to the exit, distance from monsters, and low risk-map values. This ensures that in cases where the stochastic policy points into danger, the agent overrides it with a conservative fallback.

4) *Other Features*: Several additional techniques to improve MDP robustness:

- **Reward shaping**: Adding a distance-based shaping term ensures that the agent does not stall in open maps and always prefers paths reducing exit distance.
- **Probabilistic transitions**: By modeling action uncertainty, the agent avoids risky single-step commitments and instead favors strategies that are resilient to slipping.
- **Visualization**: For debug purposes cells are visualized by exit distance and risk intensity, helping tune parameters like decay and scaling
- **Bug fix for exits**: Early on in development a issue with going to the exit square was persistent. A small safeguard was added to check if the exit is directly adjacent, ensuring the agent always steps through it even if the policy did not explicitly select that move.

#### IV. EXPERIMENTAL EVALUATION

Describe how you tested your agent:

- What maps did you use?
- How many runs per scenario?
- Metrics tracked (survival rate, score, steps to exit).

All agents were tested using a custom test script which will run the specified variant a specified number of times and record the result of each test, whether it be a failure or success.

##### A. Variant 1

The Agent for Variant 1 was tested on three different maps with varying complexity for pathing around turns. Map 1 is the simplest, with map 3 being the most complex.

20 runs were done for each section to test the ability of the agent for multiple runs. The metrics that were tracked include the survival rate of the number of runs and the survival rate of the agent.

TABLE I  
PERFORMANCE OF THE AGENT FOR VARIANT 1 DURING TESTING

| Scenario      | Runs | Survival Rate |
|---------------|------|---------------|
| Simple Map 1  | 20   | 100%          |
| Simple Map 2  | 20   | 100%          |
| Complex Map 3 | 20   | 100%          |

##### B. Variant 2

The Agent for Variant 2 was tested on the given simple map multiple times due to the randomness of the monster being added into the environment.

TABLE II  
PERFORMANCE OF THE AGENT FOR VARIANT 2 DURING TESTING

| Scenario   | Runs | Survival Rate |
|------------|------|---------------|
| Simple Map | 50   | 100%          |

##### C. Variant 3

The Agent for Variant 3 was tested on the given simple map multiple times due to the randomness of the monster being added into the environment.

TABLE III  
PERFORMANCE OF THE AGENT FOR VARIANT 3 DURING TESTING

| Scenario   | Runs | Survival Rate |
|------------|------|---------------|
| Simple Map | 20   | 90%           |

##### D. Variant 4

The Agent for Variant 4 was tested on the given simple map multiple times due to the randomness of the monster being added into the environment.

##### E. Variant 5

The Agent for Variant 5 was tested on the given simple map multiple times due to the randomness of 2 monsters being added into the environment.

TABLE IV  
PERFORMANCE OF THE AGENT FOR VARIANT 4 DURING TESTING

| Scenario   | Runs | Survival Rate |
|------------|------|---------------|
| Simple Map | 20   | 100%          |

TABLE V  
PERFORMANCE OF THE AGENT FOR VARIANT 5 DURING TESTING

| Scenario   | Runs | Survival Rate |
|------------|------|---------------|
| Simple Map | 20   | 100%          |